

3. Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as `ps`, `top`, and `/proc`. Document the observations.

```
nishanth@LAPTOP-CU1RFQM x + v
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    printf("Parent process started\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("\nChild process created\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("Child is going to sleep (Waiting/Sleeping state)...\n");
        sleep(10);

        printf("Child is running again\n");
        printf("Child process terminating\n");
    }
    else
    {
        // Parent process
        printf("\nParent process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent is waiting for child...\n");
        wait(NULL);

        printf("Child completed\n");
        printf("Parent process terminating\n");
    }

    return 0;
}
```

32,1 Top

```
nishanth@LAPTOP-CU1RFQM x + v
    printf("Parent PID: %d\n", getppid());

    printf("Child is going to sleep (Waiting/Sleeping state)...\n");
    sleep(10);

    printf("Child is running again\n");
    printf("Child process terminating\n");
}
else
{
    // Parent process
    printf("\nParent process\n");
    printf("Parent PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);

    printf("Parent is waiting for child...\n");
    wait(NULL);

    printf("Child completed\n");
    printf("Parent process terminating\n");
}

return 0;
}
```

```
nishanth@LAPTOP-CU1RFQML:~$ vi program3.c
nishanth@LAPTOP-CU1RFQML:~$ gcc program3.c -o program3
nishanth@LAPTOP-CU1RFQML:~$ ./program3
Parent process started
Parent PID: 1107
```

```
Parent process
Parent PID: 1107
Child PID: 1108
Parent is waiting for child...
```

```
Child process created
Child PID: 1108
Parent PID: 1107
Child is going to sleep (Waiting/Sleeping state)...
vChild is running again
Child process terminating
Child completed
Parent process terminating
```